

第09课：动态规划

线性 DP · 数字三角形 · 空间优化 · 最大子段和

一、学习目标

- 理解 DP 三要素：状态、转移方程、初始条件
- 掌握数字三角形路径最大和
- 学会滚动数组/空间优化
- 区分 DP 与记忆化搜索

二、核心知识点

动态规划：将问题分解为重叠子问题，存子问题解避免重复计算；通常按阶段填表。

数字三角形：`dp[i][j] = max(dp[i-1][j-1], dp[i-1][j]) + a[i][j]`；从顶到底或从底到顶均可。

空间优化：若 dp[i] 只依赖 dp[i-1]，可用一维数组滚动：`dp[j] = max(dp[j-1], dp[j]) + a[i][j]`。

最大子段和：`dp[i] = max(a[i], dp[i-1]+a[i])`；或维护 min_prefix 用前缀和优化。

记忆化：DFS + 缓存，如斐波那契；与递推 DP 等价，写法不同。

三、常识与技巧

- 定义状态要想清楚：dp[i] 或 dp[i][j] 表示什么。
- 初始化：dp[0][0]=0，不可达状态设为 -INF。
- 填表顺序：保证计算 dp[i] 时所需子状态已算完。
- 输出方案：另开 path 数组记录决策，或从终态反推。

四、参考源码文件

本课内容整理自竞赛题库文件夹 2026fare，对应源码：动态规划-P1216数字三角形.cpp、动态规划-空间优化-P1216数字三角形2.cpp、前缀和-动态规划-滑动窗口-P1115最大子段和.cpp、dfs-记忆化-P1962.cpp

五、代码示例与注释

示例 1：数字三角形 (P1216)

```
memset(dp, -0x3f, sizeof(dp));
dp[0][0] = dp[0][1] = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        cin >> a[i][j];

for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        dp[i][j] = max(dp[i-1][j-1], dp[i-1][j]) + a[i][j];
ll ans = -1e18;
```

```
for (int j = 1; j <= n; j++)
    ans = max(ans, dp[n][j]);
cout << ans;
```

示例 2: 最大子段和 DP

```
// dp[i] = ■ a[i] ■■■■■■■■
ll dp = 0, ans = -1e18;
for (int i = 1; i <= n; i++) {
    dp = max((ll)a[i], dp + a[i]);
    ans = max(ans, dp);
}
cout << ans;
```

六、配套练习题

- P1216 数字三角形
- P1115 最大子段和
- P1962 斐波那契 (记忆化)

七、本课小结

本课「动态规划」涵盖 5 个核心概念、2 段代码示例，建议结合 2026fare 文件夹中对应 .cpp 文件动手练习，并在 OJ 平台完成配套题目巩固。